SeqMobile: An Efficient Sequence-Based Malware Detection System Using RNN on Mobile Devices

Ruitao Feng*†‡, Jing Qiang Lim†‡, Sen Chen§‡, Shang-Wei Lin‡, and Yang Liu‡
‡School of Computer Science and Engineering, Nanyang Technological University, Singapore
§College of Intelligence and Computing, Tianjin University, China

***Abstract*—With the proliferation of Android malware, the demand for an effective and efficient malware detection system is on the rise. The existing device-end learning based solutions tend to extract limited syntax features, such as permissions and API calls, to meet a certain time constraint of mobile devices. However, unlike sequence-based features, syntax features lack the semantics which can represent the potential malicious behaviors and further result in more robust model with high accuracy for malware detection.**

In this paper, we propose an efficient Android malware detection system, named SeqMobile, which adopts behavior-based sequence features and leverages customized deep neural networks on mobile devices instead of the server end. Different from the traditional sequence-based approaches on server end, to meet the performance demand on mobile devices, SeqMobile accepts three effective performance optimization methods to reduce the time of feature extraction and prediction. To evaluate the effectiveness and efficiency of our system, we conduct experiments from the following aspects 1) the detection accuracy of different recurrent neural networks (RNN); 2) the feature extraction performance on different mobile devices, and 3) the detection accuracy and prediction time cost of different sequence lengths. The results unveil that SeqMobile can effectively detect malware with high accuracy. Moreover, our performance optimization methods have proven to improve the performance of training and prediction by at least twofold. Additionally, to discover the potential performance optimization from the state-of-the-art TensorFlow model optimization toolkit for our sequence-based approach, we also provide an evaluation on the toolkit, which can serve as a guidance for other systems leveraging on sequence-based learning approach. Overall, we conclude that our sequence-based approach, together with our performance optimization methods, enable us to efficiently detect malware under the performance demands of mobile devices.

***Index Terms*—Malware detection, Sequence feature, Deep neural network, Mobile platform, Performance**

I. Introduction

Smartphones have revolutionized our lives for the better in some ways. Besides calling and sending text messages, people are using these devices to watch movies, perform banking transactions [1], [2], read the news, etc. It is undeniable that these smartphones have yielded many benefits for society, allowing millions of people to stay connected through the Internet. Consequently, it has also drawn the attention of malware authors to disseminate their malware on the application markets (e.g., Google Play Store) [3]. However, unlike the App Store for Apple iOS, the protocol for uploading an application on the Android application market is not that stringent. Therefore, there is a demand for an effective malware detection system running on the device to address the above security problem.

Currently, majority of the machine learning-based malware detection systems performed their analysis on the server side [4], [5], until Drebin [6], which is a lightweight method for detection of Android malware that enables identifying malicious applications using machine learning directly on the smartphone. Recently, researchers have been looking at ways to implement effective solutions with deep learning techniques on the device-end. Due to the performance limitations on mobile devices, researchers tend to extract limited syntax features to meet certain time constraints [7], [8]. Although using syntax features without semantics (e.g., order, position) has achieved a relatively high accuracy, it will consequently fail to maintain a more robust detection system by providing necessary information to represent certain malicious behaviors. Therefore, there is a demand to research into ways to effectively represent meaningful and robust features such that it contains more semantics on the malicious behaviors with limited performance overhead on mobile devices. Unlike syntax feature based learning approaches, the feature input for sequence-based learning approaches provides not only the existence of each determined syntax feature, but also represents the semantics corresponding to certain behavior patterns [9]–[11]. However, due to the complexity of sequence-based feature, traditional server-end sequence-based learning approaches failed to satisfy the demand of run-time performance when it comes to detection on mobile devices. To address these problems, we intend to provide an efficient sequence-based malware detection system using deep learning on mobile devices.

In this paper, we propose SeqMobile, which adopts behavior-based sequence features and customized deep neural networks to provide an effective and efficient malware detection service on Android devices. To enhance the performance of SeqMobile, we propose a series of performance optimization methods that can effectively reduce the training and prediction time for sequence-based approaches. In the experiments, we first summarize and propose 8 feature categories (e.g., combination of permissions, intent filters, API sequence and intent sequence) and investigate their corresponding performance with different deep neural networks. We then perform an evaluation using accuracy and prediction

*Ruitao Feng is the corresponding author.
†These two authors contributed equally to this paper.

time as metrics to decide on a suitable network configuration. After that, we accept the pre-trained model that yields the best results and deploy it onto Android devices. To ensure that our pre-trained models are compatible with Android device, we convert our pre-trained models into lightweight TensorFlow Lite models. In our proposed system, we prioritize time cost over accuracy such that lower-end devices can choose to trade off less than 1% of the classification accuracy for lower prediction time. Overall, through our performance optimization methods, SeqMobile can achieve a relatively higher classification accuracy (i.e., 97.85%) as well as lower feature extraction and prediction time cost (i.e., <5s).

In this paper, we make the following contributions:

- We propose an efficient sequence-based malware detection system, which adopts behavior-based sequence feature and customized deep neural network to provide an effective and efficient malware detection service on Android devices.
- We present a systematic approach to directly extract the semantic feature sequence, which can provide information of certain malicious behaviors, from binary files under a certain time constraint. Thereby, achieving a relatively higher classification accuracy (i.e., 97.85%).
- We propose a method to remove repetitive elements in sequences and further evaluate how it can affect the overall performance of our malware detection system. Results has shown that our removal method significantly enhances the training and prediction performance with insignificant effects on the accuracy. To our best knowledge, this is the first comprehensive study on how removing repetitive elements in sequences can affect training and prediction performance in sequence-based learning approach on mobile devices.
- We conduct an evaluation on the state-of-the-art mobile-end model optimization toolkit provided by TensorFlow for our proposed sequence-based learning approach. The evaluation results can serve as a guidance for other mobile-end sequence-based learning approaches.

II. Background

A. Sequence Representation of Application Behavior

Mostly, Android applications provide their functionalities with basic behaviors that are represented using permissions, intent, API calls and etc. However, the analysis of Android malware shows that there are high risks that those basic behaviors inside applications may be accepted as a part of malicious functionalities. For example, considering a spyware, no matter how much it hides its malicious functionality, there will still be necessary basic behaviors existing to access the private information from those devices. Thus, a semantic representation of the basic behaviors, like sequence-based feature, will be beneficial in providing the corresponding potential malicious information unlike the traditional syntax feature in learning approaches.

B. Native Code Implementation

Considering the architecture of an Android system, the functionalities in Java implementation will be executed on Dalvik Virtual Machine. Different from traditional operating systems, when facing computationally intensive tasks, significant performance problems may occur on mobile devices. To meet certain performance criteria on Android devices, developers often investigate the bottleneck of their code and attempt improve the performance by re-implementing it using native code (C and C++), which can then be invoked through the Java Native Interface (JNI) [12]. As a result, Google recommends developers to perform the heavy operations on the native-end and return the results to the Java-end through JNI.

C. Model Quantization

Popular deep learning frameworks such as TensorFlow [13] has provided a state-of-the-art model optimization toolkit [14] to help optimized pre-trained models on mobile devices. Quantization is one of the features provided in the toolkit that can reduce the model size and prediction time by reducing the precision of the parameters inside the model (e.g., float32 to float16).

D. Static and Dynamic RNN

Known for their recurrent structure and the internal mechanism that stores the information on the previous state and forward it to the next state, RNNs are preferred when it comes to sequence-based approaches. There are two types of RNN implementations namely, static and dynamic RNNs. The main difference between them is that dynamic RNN is configured to accept variable length input while static RNN only accepts fixed length input. Having said that, when a static RNN is used, padding or truncation needs to be perform on the input sequence to ensure it matches the defined input length requirement of the model. In contrast, only truncation is needed when for dynamic RNN so that the input sequence length does not exceed the defined required length.

III. Approach

A. Overview

Fig. 1 demonstrates the overview of SeqMobile, which contains learning phase and deployment phase.

Learning phase, which is done on a server, consists of *feature preparation, network training, and model conversion*. In the feature preparation step, we focus more on extracting features as sequences such that it can provide more semantics to help distinguish between malicious and benign behaviors inside Android applications. First, a set of feature dictionaries will be constructed (step ①). Next, we extract the feature sequences and use the constructed dictionaries to filter out the redundant elements (step ②). After that, we represent each element in the filtered sequence with a unique integer identifier and pass it into our proposed network for training (step ③). Once a trained model is obtained, it will be converted into a *TensorFlow Lite* [15] model (step ④), which can then

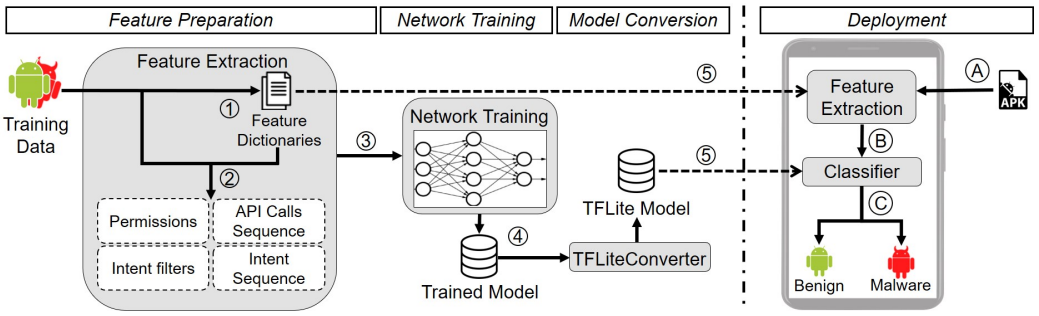


Fig. 1: Overview of SeqMobile

be loaded onto mobile device and used with the Tensorflow Lite interpreter to do inferences (step ⑤). In the midst of conversion, Tensorflow provides user an option to enable quantization, which is a technique that can further reduce the size of pre-trained model with minimal effect on the accuracy. However, we don't perform quantization in SeqMobile since our experiments in § IV-C2 shows it is more advantageous for our proposed network.

Deployment phase will first perform feature extraction from the target APK and use the constructed dictionaries from the learning phase to filter out the redundant elements (step Ⓐ), when an Android package (APK) is downloaded into the device. To ensure efficiency, a small but crucial part of the feature extraction module is implemented using native code, where a performance gain can be observed even on lower-end devices. After that, the extracted sequence will be fed into the classifier module to determine whether the target application is benign or malicious (step Ⓑ). Finally, SeqMobile will output the classification results to the user (step Ⓒ).

B. Feature Preparation

In the feature preparation step, SeqMobile mainly focuses on selecting features from the AndroidManifest.xml and DEX file (classes.dex). In order to determine the features used in SeqMobile, we perform a static analysis and select 4 kinds of informative and semantic feature sets that can potentially help to distinguish between malware and benign samples. For each feature set, we rationalize the reason of our selection with concrete examples. Besides, to remove the effect of certain elements that may not be helpful in providing information, we have built dictionaries (step ①) for the purpose of filtering out those elements. Each element in the filtered sequence will then be represented as a unique integer value so that it can be fed into our neural network for training. In addition, we also experiment with different combinations of feature sets to determine which combination yields the best accuracy with acceptable extracting performance. Based on the results, we selected 4 feature sets as our input to the neural network. The details of the experiment can be found in § IV-B1.

*1) **Feature selection**:* As a result of the strong dependency between the detection accuracy of learning-based approaches and the coverage of malicious information, the features that are able to represent more semantics in the malicious behaviors will have a higher probability to achieve a better result. Thus, based on this concept, we determine to accept two *non-sequential features*: Permission, Intent filters, and two *sequential features*: API call sequence, Intent sequence depended on our analysis against Android malware.

TABLE I: An example of API mapping

API	Description
API_0	Ljava/net/HttpURLConnection;->connect
API_1	Ljava/io/BufferedReader;->readLine
API_2	Landroid/net/Uri;->parse
API_3	Landroid/content/ContentResolver;->query

- $N^{\{Perm\}}$: Permissions are defined in the AndroidManifest file. Previous studies [16]–[18] have shown that majority of the malicious application tend to request dangerous permissions as compared to benign applications. This indicates that including permissions as a part of our feature can potentially help us distinguish malware and benign apps.
- $N^{\{Intent\}}$: Intent filters are defined inside the AndroidManifest.xml file that specifies what type of intents a component (e.g., Activity) would like to receive. Intent filter makes it possible for other applications to directly start the activity by sending out the defined intent message. An example of how malicious application abuse the intent filter is that they usually listen for the BOOT_COMPLETED intent, which is sent after successfully booting up the mobile device, to start their malicious activities [18].
- $S^{\{API\}}$: Representing API calls features in an unordered manner is usually sufficient to provide enough information on the behaviors of an Android application. However, if the API calls are represented in a sequential manner, it can provide us with additional semantics amongst the API calls. Two behaviors are defined using sequence of API calls, where $S_0^{\{API\}}$=$\{API_0, API_1, API_2, API_3\}$ and $S_1^{\{API\}}$=$\{API_2, API_3, API_0, API_1\}$. The details of each API can be found in Table I. The behavior $S_0^{\{API\}}$ represents the action of communicating to the internet followed by reading of SMS inbox messages. This is a typical behavior for instant messaging applications where they try to verify your mobile number. In contrast, $S_1^{\{API\}}$ represents the action of reading of SMS inbox messages followed by a communication to the internet, which is possibly a malicious behavior where an adversary attempts to retrieve inbox messages.
- $S^{\{Intent\}}$: Besides picking API call sequence as a part of our feature set, we also accept the sequential intent as a supplemental feature for the API call sequence with the

TABLE II: Vocabulary size of each dictionary

Dictionary	Vocabulary size
Permissions	324
Intents	262
API calls	2,288

following two reasons. Firstly, when API call sequence is included as our feature set, the function parameters are not taken into account, thus, generic API calls such as Landroid/content/Intent;->init or Landroid/content/Intent;->setAction is unable to provide information on the purpose of invocation unless the corresponding *string* parameter is included. Secondly, relying on intent filters is not sufficient as it represents what type of intent the component is looking out for. Oftentimes, intents can be omitted from the manifest file. One typical example is that malware can make use of the ACTION_CALL intent to call premium rate numbers while the user is not looking [19]. In addition, the ACTION_CALL intent does not need to be defined in the manifest file. Thus, we include the *string* parameter of the generic API calls mentioned above in a sequential order and intertwine it with the API call sequence (e.g., $S^{\{API,Intent\}}$=$\{API_1, IN_0, API_2, API_0, IN_2, API_3\}$).

*2) **Feature dictionary construction**:* Since there are certain elements that may not be related to potential malicious behaviors in the raw data (i.e., .xml and .dex files) within each feature set, a set of feature dictionaries are necessary in the feature extraction step. Thus, based on the static analysis result on potential malicious behaviors, a set of feature dictionaries has been constructed (step ①).

Permission and intent dictionary. To build the permission and intent dictionary, we refer to the Android source code which are predefined by Google developers to retrieve all the permissions and intent filter values.

API call dictionary. To build the API call dictionary, we conduct a data-driven analysis to collect API calls from more than 60,000 real-world applications and pruned the result set by removing self-defined API calls as well as uncommon third-party API calls [8].

In summary, we constructed 3 feature dictionaries and the total number of vocabulary in each dictionary can be found in Table II.

*3) **Feature extraction**:* To generate and formalize the selected features (step ②), we first propose a repetitive elements removal method, which can boost the performance of our system with little effect on the accuracy (discussed in § IV-B2), by reducing the length of feature sequence for the selected sequence-based features. Next, with the shorten feature sequence, we transform it into a numeral sequence such that it will be suitable to be fed into the neural network.

Raw string based feature representation. To extract $S^{\{API\}}$, we disassemble the DEX file and look for instructions starting with the *invoke-** opcode. At the same time, we also include the $S^{\{Intent\}}$ feature into the sequence by looking out for instructions that starts with the *const-string* opcode. For each of the instructions found, we concatenate them to form a *string* based sequence, S_d. An example of such sequence will be

TABLE III: Network architecture - Bi-LSTM

Layer	Output Shape
Embedding	(None, None, 128)
Bidirectional (LSTM)	(None, None, 512)
Batch Normalization	(None, None, 512)
GlobalMaxPooling1D	(None, 512)
Dense (ReLU)	(None, 64)
Dense (ReLU)	(None, 32)
Dense (softmax)	(None, 2)

$S^{\{API,Intent\}}$=$\{API_0, API_1, IN_1, IN_2, API_3\}$ where IN_1 and IN_2 are intent values. After that, permissions and intent filters will be extracted and matched against the corresponding dictionaries to construct the non-sequential feature, $N^{\{Perm\}}$ and $N^{\{Intent\}}$. Finally, we concatenate the extracted sequence, S_d, together with $N^{\{Perm\}}$ and $N^{\{Intent\}}$ to form the final sequence, S_f. Based on the experiment results presented in § IV-B1, we select 4 features as the final feature set namely, $N^{\{Perm\}}$, $N^{\{Intent\}}$, $S^{\{API\}}$, and $S^{\{Intent\}}$.

Repetitive elements removal. Oftentimes, there can be repeated elements in a sequence. An example of how this can occur is that a developer may define two different methods, M_1 and M_2, that execute very similar tasks. We define API_i and IN_i as API calls and intent values which are defined in the dictionaries and S_API_i as self-defined API calls not found in the API dictionary. Consider M_1= $\{IN_0, API_0, S_API_0\}$ and M_2=$\{IN_0, API_0, S_API_1, API_1\}$. After the extracted sequence is filtered against the respective dictionaries, the resulting sequence is $\{IN_0, API_0, IN_0, API_0, API_1\}$, which is basically a repetition of $\{IN_0, API_0\}$ followed by an API_1. If we view it from the perspective of a text sentiment analysis problem, having the sentence "this movie is great, this movie is great" does not change the polarity of the sentence, likewise, having repetitive elements in the sequence will have insignificant effects (§ IV-B2) on the polarity (i.e., malicious or benign). Thus, the resulting sequence after removing the repetitive elements will be $\{IN_0, API_0, API_1\}$.

Dictionary identifier assignment and integer based sequence representation. In order to transform the string based sequence into an integer sequence, we first assign each dictionary element with a unique integer identifier and store the pair in a look up table, T_l. By referring to the look up table, we represent each element in S_f with their respective integer identifier. After that, we can feed it into the embedding layer of the proposed network Table III (step ③).

C. Deep Learning Model Construction

To discover the usability of different neural networks for our selected feature sets, we present 6 basic neural networks to train our classifier (i.e., single layer LSTM/GRU, stacked LSTM/GRU, and Bi-LSTM/GRU) (details on our website [20]). By comparing the accuracy, we accept the Bi-LSTM, which yields the highest accuracy, as basic network and perform a customization on the architecture to further enhance the accuracy. The architecture of the customized Bi-LSTM network is shown in Table III.

*1) **Sequence padding**:* The length of S_f varies from one application to another, hence, we pad the input sequence to

ensure that the length is consistent with the required input length of the network, *L*. If the length of S_f is shorter than *L*, we perform post-zero padding to S_f till it reaches *L*. If the length of S_f is longer than *L*, we truncate S_f till *L*.

*2) **Customized deep neural network architecture**:* To train our malware classifier, we present a customized Bi-LSTM network. As shown in Table III, the first layer is an embedding layer. Due to our large vocabulary size, representing our input sequence using one-hot encoding results in a sparse vector, which is not memory efficient during training and prediction. By incorporating an embedding layer, it can help to reduce the dimensionality of our feature vectors where each element of our integer input sequence is represented as a lower dimension fixed sized vector. The second layer is a Bi-LSTM layer, which has the ability to preserve information from both the forward and backward along the sequence, thus allowing the network to understand the contextual information better and results in a more comprehensive learning of the problem. The third layer is a batch normalization layer, it has been shown that incorporating a batch normalization layer to the network can reduce the internal covariate shift [21], [22]. After that, a global max-pooling function is used to capture the most important factor. Then, the output from the global max-pooling function will pass through the 2 fully connected layers with Rectified Linear Unit (ReLU) activation function followed by a fully connected layer with a softmax activation function. Finally, the output from the last fully connected layer determines which class the input sequence belongs to.

D. Model Conversion and Quantization

In order to make our pre-trained model compatible with mobile devices, we convert the pre-model into a TensorFlow Lite model using TFLiteConverter [23]. We have also conducted experiments and decided not to perform quantization even though it has advantages such as prediction time and model size reduction. By not applying quantization to our model, we are able to achieve dynamic prediction time where the prediction time cost is dependent on the extracted sequence length (details in § IV-C2).

E. Real Time Detection System

Before conducting a real-time detection on device, the feature dictionaries and converted model will be loaded into the feature extraction and classifier modules respectively (step ⑤). Once a new APK file is received, SeqMobile first performs feature extraction on the target APK with the constructed dictionaries from the learning phase (step Ⓐ). To improve the overall performance of SeqMobile, we directly extract the selected features from the binary files and perform repetitive sequence removal such that the sequence length can be reduced. In addition, we also optimized our feature extraction module by incorporating some native code in the implementation. Next, we perform truncation to the input sequence but not padding. Results from our experiments shows that without padding, we can not only achieve the best accuracy, but also a shorter prediction time. Finally, the classifier module will determine from the extracted features whether the Android application is malicious or benign.

TABLE IV: Dataset

Class	Source	Quantity
Malicious	Contagio	327
Malicious	Drebin	5,339
Malicious	Genome	1,253
Malicious	Pwnzen	774
Malicious	Virusshare	14,797
Benign	Google Play Store	22,490
Total Malicious	-	22,490
Total Benign	-	22,490
Total	-	44,980

IV. Experiments

In this section, we present four sets of experimental studies. We aim to determine: (1) the detection performance of different networks across different combination of feature categories; (2) the training and prediction performance gain through our repetitive elements removal method; (3) the feature extraction performance across different mobile devices; and (4) the performance comparison between the quantized and non-quantized dynamic RNN model. Finally, we briefly compare the performance of our approach with two other previous work [7], [8].

A. Experiment Environment and Dataset

Environment. All experiments are conducted on an Ubuntu server with Intel Xeon E5-2699 V3 CPUs, NVIDIA GeForce RTX 2080 Ti GPU and 6 different Android devices, which consists of 3 flagship devices (i.e., Samsung Note10+, S10+, S9+), 1 common device (i.e., Samsung S7), and 2 low-end devices (i.e., Samsung J2 Pro and HTC ONE A9).

In our server-end tasks, Java is our choice of language for implementing the feature extraction module and TensorFlow 2 [13] is chosen as our deep learning framework. For feature extraction, we use additional tools such as *dexdump* [24] and AXMLPrinter2 [25]. As for the TensorFlow Lite converter, a specific TensorFlow nightly build (2.2.0.dev20200430) [26] which supports our proposed network architecture is used.

Dataset. To evaluate SeqMobile, we collected 44,980 Android applications samples which can be divided into two classes; benign and malicious. We crawled the benign samples from Google Play Store, while the malicious set is composed of samples from different sources such as Drebin [6], Genome project [27], Contagio Mobile [28], VirusShare [29], and Pwnzen Infotech Inc. [4], [5]. The breakdown of the dataset is shown in Table IV. To split our dataset, we randomly select 70% of the samples from each class for training, 15% for validation and 15% for testing.

B. Effectiveness Evaluation of Feature Selection, Deep Neural Networks, and Repetitive Pattern Removal in Feature Preparation and Network Training Phases

We evaluate the effectiveness of SeqMobile in the learning phase from the following aspects: (1) the accuracy across different feature categories, length, and deep neural networks

TABLE V: Detection results of best feature category combinations across difference networks

Network	Feature Combination	Sequence Length	Accuracy	Precision	Recall
Single LSTM	$N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API\}}$	600	96.47%	97.66%	95.23%
Single GRU	$N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API\}}$	1,500	96.78%	96.99%	96.56%
Stacked LSTM	$N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API\}}$	600	96.49%	97.86%	95.05%
Stacked GRU	$N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API\}}$	1,500	96.90%	97.42%	96.35%
Basic Bi-LSTM	$N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API\}}$	1,500	96.98%	97.51%	96.41%
Basic Bi-GRU	$N^{\{Perm\}}$ and $S^{\{API,Intent\}}$	1,900	96.75%	97.47%	96.00%
Customized Bi-LSTM	$N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API,Intent\}}$	1,700	97.85%	97.89%	97.81%
Customized Bi-GRU	$S^{\{API\}}$	1,500	97.67%	98.11%	97.21%

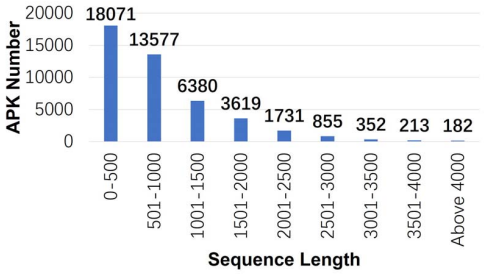


Fig. 2: Distribution of extracted sequence length

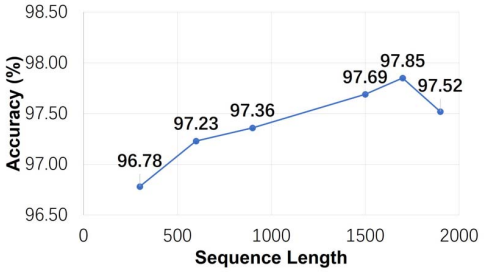


Fig. 3: Accuracy change across different sequence length on $N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API,Intent\}}$

and (2) the effect on training accuracy and performance brought by our repetitive pattern removal method.

1) Accuracy comparison across feature categories and deep neural networks: To determine the best training configuration, we set up an experiment across three aspects (i.e., feature categories, sequence length, and network types). To find out the most suitable network, we first evaluate the detection accuracy of 6 different basic network configurations across 8 different feature categories (details on our website [20]), which apply $S^{\{API\}}$ as the basic feature and combined with $N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{Intent\}}$ respectively. In addition, as shown in Fig. 2, majority of the APKs in our dataset have sequence length ranging up to 2,000. Thus, we also train each network across different sequence length to determine the appropriate sequence length that yields the best accuracy. To choose the range of sequence length to experiment on, we progressively increase the sequence length until no significant improvement on the accuracy can be observed. In total, according to the distribution of extracted sequence length, we select 6 different values ranging from 300 to 1,900 to train the networks. As shown in Table V, the basic Bi-LSTM network achieves the highest accuracy (i.e., 96.98%) out of the other basic networks. To further improve the detection accuracy, we customize the basic Bi-LSTM to our best effort such that there is a significant

TABLE VI: Average sequence length

Dataset	Original Length	After Removal
Benign	2,789	1,060
Malware	1,249	598

improvement on the accuracy. We first progressively increase the number of parameters (i.e., increase LSTM units and add fully connected layer) in the network. Consequently, a longer training duration can be observed due to the increased network parameters. Thus, we added a batch normalization followed by *GlobalMaxPooling1D* layer to reduce the training duration. Our experiment results shows that by configuring the network as shown in Table III, the feature set combination that achieves the best accuracy is $N^{\{Perm\}}$, $N^{\{Intent\}}$, and $S^{\{API,Intent\}}$, which achieves an accuracy of 97.85% when the sequence length is 1,700 (shown in Fig. 3).

2) Effect of repetitive pattern removal: We conduct experiments to investigate how removing of repetitive sequence will benefit the performance of the system. In the experiment, we use the following metrics: (1) accuracy and (2) training duration to evaluate our method. The results show that (1) removing the repetitive pattern in a sequence has insignificant effects on the learning ability of the neural network; (2) it can help improve the training and prediction performance in terms of time cost.

Average number of removed repetitive elements. To show the necessity of our repetitive elements removal method, we first calculate the number of repetitive elements removed in each dataset (i.e., benign and malware) to get a rough estimation of the proportion being removed by comparing with the original average sequence length. As shown in Table VI, the average sequence length is reduced by approximately 62% (1,060 vs. 2,789) in the benign dataset. Similarly, the average sequence length is reduced by approximately 52% for the malware dataset. When taking into account for both datasets, the sequence length is reduced by 57% on average. Thus, based on the results, we take 0.6 as an approximate proportion of the repetitive elements in the original sequences and define equation (1) to calculate the estimated original sequence length for each non-repetitive sequence length.

$$Seq_{original} = \frac{Seq_{non-repetitive}}{1 - 0.6} \quad (1)$$

Accuracy comparison between non-repetitive and original sequence. To discover the effect of our repetitive elements removal method on model accuracy, we define the following 2 models, M_1, which adopts the sequences without repetitive

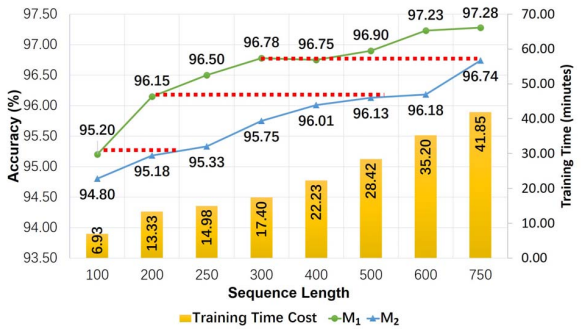


Fig. 4: Training time across different sequence length and accuracy comparison between M_1 and M_2

elements as its input, $Seq_{non-repetitive}$, and M_2, which is trained using the original sequences that contain repetitive elements, $Seq_{original}$, and train them with sequence length ranging from 100 to 750. From the previous experiment, the input sequence length is reduced by 57% on average after removing repetitive elements, as a result, with the same sequence length, $Seq_{non-repetitive}$ will provide more information than $Seq_{original}$. To compare the accuracy of M_1 and M_2, we use the equation in (1) to estimate the corresponding original sequence length for each $Seq_{non-repetitive}$, and assign them as a control group. For example, if the accuracy of M_1 at sequence length 200 is 96.15%, we will apply equation (1) to calculate the length of $Seq_{original}$ (i.e., 500). From the line chart in Fig. 4, the accuracy for M_2 is 96.13% when the sequence length is 500, which is very close to the accuracy for M_1 (i.e., 96.15%) at sequence length 200. Similarly, the accuracy at M_1 when the sequence length is 300, is very close to the accuracy of M_2 when the sequence length is at 750 (96.78% vs. 96.74%). This shows that by removing repetitive elements in a sequence, it will have very little effect on the learning ability of the network.

Performance improvement in network training. To find out the performance improvement of our repetitive elements removal method in the network training phase, we also provide a comparison on training time between each grouped models with non-repetitive and original sequences respectively. From the histogram in Fig. 4, the training time for M_2 is 28.42 minutes when the sequence length is 500. Comparing with the time for M_1 (i.e., 13.33 minutes) at sequence length 200, our repetitive elements removal method improves the training performance by around 53.1%. Similarly, comparing the training time for M_1 when the sequence length is 300, we observe a much shorter training time for M_2 when the sequence length is at 750 (17.40 minutes vs. 41.85 minutes). This provides a strong evidence for proving that our repetitive elements removal method benefits a lot in the training performance.

C. Performance Evaluation and Optimization of Feature Extraction and Detection on Android Mobile Devices

We evaluate the performance of our device-end modules (i.e., feature extraction and prediction modules) separately. For the feature extraction module, we conduct experiments to determine (1) the performance of extraction time on mobile devices, by comparing the extraction time of our final feature combination (i.e., $N^{\{Perm\}}$, $N^{\{Intent\}}$ and $S^{\{API,Intent\}}$) across 6 different APK sizes; and further evaluate (2) the performance gain of the feature extraction module where certain parts are rewritten in native code. For the prediction module, we assess the performance gain across 2 aspects, which are input sequence and deployed model configuration.

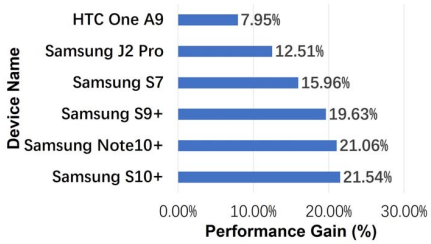


Fig. 5: Performance gain with JNI implementation

1) Performance comparison of feature extraction between implementing with JNI and Java language on Android devices: To discover the potential performance optimizations in feature extraction on real devices, we implement part of the feature extraction module that involves *string* manipulation in C++ and use JNI to interact with the native implementation. Based on the previous experiment, we select the feature set combination that yields the highest accuracy (i.e., $N^{\{Perm\}}$, $N^{\{Intent\}}$ and $S^{\{API,Intent\}}$) and measure the time cost between the Java and JNI implementation. The APKs used in this experiment are handpicked, with file sizes ranging from 5MB to 50MB. We then calculate the performance gain between the implementations using equation (2).

$$Performance\ gain = (1 - \frac{Timecost_{JNI}}{Timecost_{Java}}) \times 100\% \quad (2)$$

As shown in Fig. 5, the JNI implementation can improve the performance of the feature extraction module by approximately 21% on flagship phones. Even on low-end devices such as HTC ONE A9, a 7.95% increase in performance can be observed.

A detailed chart of the feature extraction (JNI implementation) time cost across different devices is shown in Fig. 6. We observe that flagship phones such as Samsung S10+, Samsung Note10+ and Samsung S9+, are able to extract the features faster than common and lower end devices (Samsung S7, Samsung J2 Pro, and HTC ONE A9). The time cost to extract 5MB applications on the flagship phones is between 0.144s and 0.183s. While on common and low-end devices, it takes approximately 0.301s to 0.641s to extract the features. Similarly, for 50MB applications, the flagship phones outperform the low-end devices (3.701s vs. 16.120s).

2) Performance optimizations in prediction on Android devices: To provide a guidance for improving the performance of sequence-based learning approaches, which may also accept RNN as their computational layer, in the real-time prediction on mobile devices, we conduct two experiments from different aspects. (1) We investigate the performance optimization, which brought by our proposed repetitive pattern removal method, by comparing the prediction time between the original

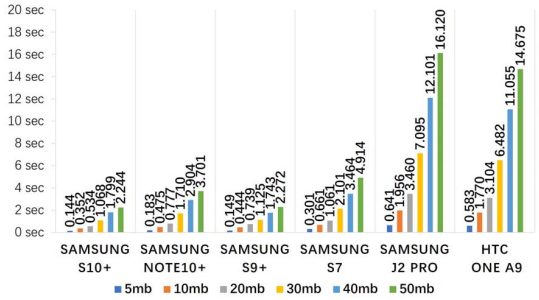


Fig. 6: Feature extraction (JNI implementation) time cost for different APK sizes across different devices

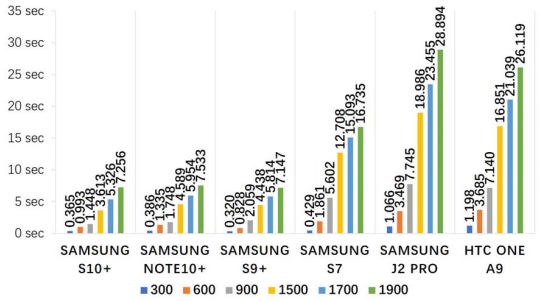


Fig. 7: Prediction time cost for different sequence lengths across different devices

and non-repetitive sequence inputs. (2) We evaluate the state-of-the-art model optimization toolkit provided by TensorFlow for our sequence-based approach. In the second experiment, we first conduct a preliminary investigation to determine the characteristic differences (i.e., model size, input requirements, etc.) between the quantized and non-quantized models. Based on the findings, we further investigate the influence of quantization by comparing the time taken to predict sequences of different lengths between the quantized and non-quantized dynamic RNN models.

Repetitive pattern removal. To discover the effect of our proposed repetitive element removal method in prediction on real devices, we conduct an experiment to measure the prediction time for 6 different sequence lengths across 6 different devices. In this experiment, we randomly pick 5 APKs from each sequence length category (i.e., 300, 600, 900, 1,500, 1,700, and 1,900).

As shown in Fig. 7, flagship devices such as Samsung S10+ takes approximately 2.72 times longer (0.365s vs. 0.993s) to predict a sequence that is double the original length (e.g., 300 vs. 600). Similarly for low-end devices such as Samsung J2 Pro and HTC ONE A9, the prediction time is approximately 3.25 times longer (1.198s vs. 3.685s) to predict a sequence that is double the original length. We observe that it takes at least a twofold increase in prediction time cost to predict a sequence that is twice the original length. Based on our experiment in § IV-B2, we observe that our method reduces the sequence length by approximately 57%, which directly translates to an improvement in prediction time of at least twofold. Apart from the removal method, different from the traditional sequence-based learning approaches that work on

TABLE VII: Non-quantized and quantized model size comparison

Seq Len.	Non-quantized(MB)	Quantized(MB)
300	7.62	2.97
600	7.62	3.12
900	7.62	3.27
1,500	7.62	3.56
1,700	7.62	3.66
1,900	7.62	3.75

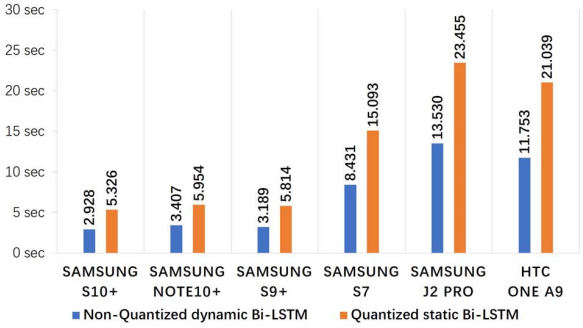


Fig. 8: Average prediction time between non-quantized dynamic Bi-LSTM and quantized static Bi-LSTM across different devices

server end, approaches on real devices, which may have a strong performance limitation, should take the input sequence length of their defined networks as an important factor to optimize their real-time performance on device.

Model quantization. To investigate the potential performance oriented influence and their corresponding factors of quantization in our proposed system, we first conduct a preliminary investigation to figure out the main differences between a quantized and non-quantized model. In this experiment, we first compare the model size of the quantized and non-quantized models across different sequence length. As shown in Table VII, when the sequence length is 300, the model size is reduced by 61% (7.62MB to 2.97MB). However, as the sequence length increases to 1,900, the model size is reduced by approximately 50.8% (7.62MB to 3.75MB). From the results, it is evident that the size of the quantized model is dependent on the sequence length (i.e., longer sequence length will constitute to a larger file size), while the size for the non-quantized model remains constant across different sequence length.

Next, we also conduct experiments to compare the accuracy between the quantized and non-quantized model against the pre-trained model. The accuracy of quantized and non-quantized model remains 97.85%, which is same as the pre-trained model. Otherwise, we also notice that if the input requirements of a network is designed to allow variable length input (e.g., dynamic RNN), quantizing the model will remove the flexibility for allowing variable length input (i.e., inputs will be static fixed length). On the contrary, the input requirements remain unchanged if quantization is not applied. In the event where a quantized model is deployed, padding or truncation is required to ensure that the input sequence is of a certain length.

Dynamic RNN. Since our dataset consists of sequences of varying lengths, designing our network to use dynamic RNN will be beneficial in the deployment phase. Based on the

findings from the preliminary investigation, we observe that quantizing our model is equivalent to using a static RNN model, hence, we refer to the quantized model as quantized static Bi-LSTM model and non-quantized dynamic Bi-LSTM for the non-quantized model.

We conduct an experiment to determine if quantizing our dynamic RNN will improve the performance of our system. In this experiment, we apply the same test set from our previous experiment, which contains 30 APKs with 6 different sequence length. Next, we accept the pre-trained model with the highest accuracy to compare the average prediction time between the quantized static Bi-LSTM and non-quantized dynamic Bi-LSTM models. As shown in Fig. 8, we observe that the average prediction time for the non-quantized dynamic Bi-LSTM model is much faster than the quantized static Bi-LSTM model. On flagship phones such as Samsung S10+, the average time taken to predict an extracted sequence is 2.928s when using the non-quantized dynamic Bi-LSTM model. On the contrary, it takes 5.326s on average to predict with the quantized static Bi-LSTM model. The average time cost is lowered by approximately 45% across all devices. The root cause of this observation is expected as padding or truncation is required for the quantized model to make the input length consistent, i.e., 1,700, which constitutes to a higher average prediction time. Based on the results, we decide not to quantize our pre-trained model as it brings about more advantages for our proposed network. The allow for variable length input in the non-quantized dynamic Bi-LSTM model has enable us to achieve a dynamic prediction time as no padding is required (i.e., the time cost per prediction is dependent on the extracted sequence length). On the contrary, a fixed length input is required for the quantized static Bi-LSTM model, where each sequence is padded or truncated to a certain length. By doing so, it constitutes to a consistently higher time cost per prediction. Although the model size is reduced by half (7.62MB to 3.66MB) after quantization, compromising time cost for model size is not a feasible option for our performance-sensitive malware detection system. Hence, for any sequence based performance-sensitive learning approaches, which uses RNN as the basic computational layer, dynamic RNN is currently an important option to optimize their system performance instead of quantization, which is widely adopted.

Currently, TensorFlow does not support quantization for models that accept variable length input (e.g., dynamic RNN) [30]. During quantization, a "reshape" operation is added internally to ensure that the input requirement is of fixed length. Thus, padding or truncation of the sequence is required, which results in the dynamic RNN model being indirectly converted into a static RNN model.

D. Comparison between previous work and SeqMobile

We briefly compare SeqMobile against two other previous works (i.e., MobiDroid [7] and MobiTive [8]). We use our results for Samsung S10+ as a baseline to benchmark against Nexus 6P from MobiDroid and Huawei P30 from MobiTive. As shown in Table VIII, even though the features used in the other two systems are similar (e.g., API calls and manifest properties), our sequence-based approach, which contains additional semantics, is able to achieve a much higher accuracy (i.e., 97.85%). Although there is an improvement in time cost when comparing to MobiDroid (4.16s vs. 17.76s), our time cost is still approximately 10 times higher (4.16s vs. 0.46s) when compare to MobiTive. Despite our high time cost, with the additional semantics from the sequence-based features, our accuracy is higher than MobiTive (96.75% vs. 97.85%). Also, the study from MobiTive [8] shows a trend of mobile hardware performance improving over the years, we strongly believe that our approach can achieve a significantly lower time cost in the upcoming years, making the accuracy of the detection system a much more important factor.

TABLE VIII: Comparison of SeqMobile against previous work

Systems	Features used	Accuracy(%)	Time cost(s)
MobiDroid	Opcode sequence API calls 3 Manifest properties	97.35%	17.76
MobiTive	API calls 3 Manifest properties	96.75%	0.46
SeqMobile	API and intent sequence 2 Manifest properties	97.85%	4.16

V. Related Work

In this section, an overview of the current deep learning malware detection approaches will be presented. Generally, these approaches can be categorized into server end and device end approaches.

A. Server-end Approach

Malware detection on server end are usually efficient due to the computational resources available. To summarize, there are approaches [31]–[36] that extract features (e.g., permissions and API calls) from XML and DEX files and represent them as feature vectors to uncover potential malicious behaviors. There are also approaches [9]–[11], [37] that represent features (e.g., API calls, opcode, system calls) as sequences to detect malware. Also, some approaches [38]–[41] use the additional computation resources to their advantage and analyze complex features such as control flow graphs. While server end approaches achieve great success in detecting malware, it also incurs network transmission overhead for sending the required files to the server.

B. Device-end Approach

Device-end solutions are often performance-sensitive systems that can effectively detect malware under a certain time constraint. Given the performance limitations of Android devices, such device-end solutions usually have limited resources to extract and analyze complex features to classify the applications. Thus, only limited features can be used to detect malware. Feng et al. proposed MobiDroid [7], a performance-sensitive malware detection system, which represents features (e.g., opcode sequence, API calls, and manifest properties) as feature vectors and leverages deep learning algorithms to help detect malware. Another study by Feng et al. [8], uses a different method to directly extract features (e.g., API calls and

manifest properties) from binary files to perform classification with a low overhead time. This paper studies the device-end performance of sequence-based malware detection and propose performance optimization methods for sequence-based learning approaches. Closest to our device-end sequence-based approach, Elmouatez et al. proposed Maldozer [42], which uses API sequences to detect Android malware families. Differently, they did not consider the performance on devices as a first order factor in the approach. Existing device-end approaches mainly focus on extracting limited feature sets to meet the time constraints. With the rapid advancement in technology, the performance of Android devices is improving every year [8]. Thus, studies on device-end malware detection systems is much needed.

VI. Conclusion

In this paper, we have investigated the effectiveness of using sequence-based learning approach together with performance optimization methods to detect malicious applications on device end. The evaluation results show that our approach achieves a high accuracy (97.85%) and a reasonable detection time on flagship phones. Moreover, we have also provided a guidance on the state-of-the-art TensorFlow model optimization toolkit for device-end sequence-based approaches.

Acknowledgments

This work was supported by Singapore Ministry of Education Academic Research Fund Tier 1 (Award No. 2018-T1-002-069), the National Research Foundation, Prime Ministers Office, Singapore under its National Cybersecurity R&D Program (Award No. NRF2018 NCR-NCR005-0001), the Singapore National Research Foundation under NCR Award Number NSOE003-0001, NRF Investigatorship NRFI06-2020-0022, the National Research Foundation, Prime Ministers Office, Singapore under NCR Award Number NRF2018NCR-NSOE004-0001, the National Natural Science Foundation of China (No. 61902395). We gratefully acknowledge the support of NVIDIA AI Tech Center (NVAITC).

References

[1] S. Chen, T. Su, L. Fan, G. Meng, M. Xue, Y. Liu, and L. Xu, "Are mobile banking apps secure? what can be improved?" in *FSE*, 2018.
[2] S. Chen, L. Fan, G. Meng, T. Su, M. Xue, Y. Xue, Y. Liu, and L. Xu, "An empirical assessment of security risks of global android banking apps," in *ICSE*, 2020.
[3] C. Tang, S. Chen, L. Fan, L. Xu, Y. Liu, Z. Tang, and L. Dou, "A large-scale empirical study on industrial fake apps," in *ICSE*, 2019.
[4] S. Chen, M. Xue, Z. Tang, L. Xu, and H. Zhu, "Stormdroid: A streaminglized machine learning-based system for detecting Android malware," in *AsiaCCS*, 2016.
[5] S. Chen, M. Xue, L. Fan, S. Hao, L. Xu, H. Zhu, and B. Li, "Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach," *Computers & Security*, 2018.
[6] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, K. Rieck, and C. Siemens, "Drebin: Effective and explainable detection of Android malware in your pocket." in *NDSS*, 2014.
[7] R. Feng, S. Chen, X. Xie, L. Ma, G. Meng, Y. Liu, and S.-W. Lin, "MobiDroid: A performance-sensitive malware detection system on mobile platform," in *ICECCS*, 2019.
[8] R. Feng, S. Chen, X. Xie, G. Meng, S.-W. Lin, and Y. Liu, "A performance-sensitive malware detection system using deep learning on mobile devices," *arXiv preprint arXiv:2005.04970*, 2020.
[9] N. McLaughlin, J. Martinez del Rincon, B. Kang, S. Yerima, P. Miller, S. Sezer, Y. Safaei, E. Trickel, Z. Zhao, A. Doupé *et al.*, "Deep android malware detection," in *CODASPY*, 2017.
[10] R. Nix and J. Zhang, "Classification of Android apps and malware using deep neural networks," in *2017*, IJCNN.
[11] Q. Jerome, K. Allix, R. State, and T. Engel, "Using opcode-sequences to detect malicious Android applications," in *2014*, ICC.
[12] Java native interface (JNI) framework. [Online]. Available: https://docs.oracle.com/javase/7/docs/technotes/guides/jni/spec/jniTOC.html
[13] Tensorflow. [Online]. Available: https://www.tensorflow.org/
[14] Model optimization. [Online]. Available: https://www.tensorflow.org/lite/performance/model_optimization
[15] Tensorflow Lite. [Online]. Available: https://www.tensorflow.org/lite/
[16] S. Verma and S. Muttoo, "An Android malware detection framework-based on permissions and intents," *Defence Science Journal*, 2016.
[17] H. K. et al., "Detecting and classifying Android malware using static analysis along with creator information," *International Journal of Distributed Sensor Networks*, 2015.
[18] A. Feizollah, N. Anuar, R. Salleh, G. Suarez-Tangil, and S. Furnell, "Androdialysis: Analysis of Android intent effectiveness in malware detection," *Computers & Security*, vol. 65, 2017.
[19] MouaBad: Pocket Dialing for Profit. [Online]. Available: https://blog.lookout.com/mouabadp-pocket-dialing-for-profit
[20] Overview of SeqMobile. [Online]. Available: https://sites.google.com/view/seqmobile/home
[21] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," ser. ICML, 2015.
[22] T. Cooijmans, N. Ballas, C. Laurent, Ç. Gülçehre, and A. Courville, "Recurrent batch normalization," *arXiv preprint arXiv:1603.09025*, 2016.
[23] Tensorflow Lite Converter. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/lite/TFLiteConverter
[24] Android SDK build tools 28. [Online]. Available: https://aur.archlinux.org/packages/android-sdk-build-tools-28/
[25] A parser for binary XML files. [Online]. Available: https://code.google.com/archive/p/android4me/downloads
[26] tensorflow-nightly build 2.2.0.dev20200430. [Online]. Available: https://libraries.io/pypi/tf-nightly/2.2.0.dev20200430
[27] Y. Zhou and X. Jiang, "Dissecting Android malware: Characterization and evolution," in *S&P*, 2012.
[28] Contagio. [Online]. Available: http://contagiominidump.blogspot.com/
[29] Virusshare. [Online]. Available: https://virusshare.com/
[30] Quantized model not supporting dynamic input length. [Online]. Available: https://github.com/tensorflow/tensorflow/issues/24607\#issuecomment-582675214
[31] K. A. Talha, D. I. Alper, and C. Aydin, "Apk Auditor: Permission-based Android malware detection system," *Digital Investigation*, 2015.
[32] Z. Aung and W. Zaw, "Permission-based Android malware detection," *International Journal of Scientific & Technology Research*, 2013.
[33] P. Rovelli and Y. Vigfusson, "PMDS: Permission-based malware detection system," in *ICISS*, 2014.
[34] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Deep Android malware detection and classification," in *ICACCI*, 2017.
[35] S. Chen, M. Xue, and L. Xu, "Towards adversarial detection of mobile malware," in *MobiCom*, 2016.
[36] L. Fan, M. Xue, S. Chen, L. Xu, and H. Zhu, "Accuracy vs. time cost: Detecting Android malware through pareto ensemble pruning," in *CCS*, 2016.
[37] X. Xiao, S. Zhang, F. Mercaldo, G. Hu, and A. K. Sangaiah, "Android malware detection based on system call sequences and LSTM," *Multimedia Tools and Applications*, 2017.
[38] Z. Ma, H. Ge, Y. Liu, M. Zhao, and J. Ma, "A combination method for Android malware detection based on control flow graphs and machine learning algorithms," *IEEE Access*, 2019.
[39] S. Alam, Z. Qu, R. Riley, Y. Chen, and V. Rastogi, "Droidnative: Automating and optimizing detection of Android native code malware variants," *Computers & Security*, 2017.
[40] Z. Xu, K. Ren, S. Qin, F. Craciun, J. Sun, and M. Sun, "Cdgdroid: Android malware detection based on deep learning using CFG and DFG," in *Formal Methods and Software Engineering*, 2018.
[41] J. Yan, G. Yan, and D. Jin, "Classifying malware represented as control flow graphs using deep graph convolutional neural network," *DSN*, 2019.
[42] E. Karbab, M. Debbabi, A. Derhab, and D. Mouheb, "Maldozer: Automatic framework for Android malware detection using deep learning," *Digital Investigation*, 2018.